

BÀI THỰC HÀNH CHAIN OF RESPONSIBILITY PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (gửi request)
 - ✓ Áp dụng Chain of Responsibility chuẩn GoF
 - ✓ Minh họa rõ luồng request đi qua nhiều handler
-

BÀI TOÁN

Hệ thống xử lý yêu cầu phê duyệt đơn hàng theo cấp độ:

-   Nhân viên: duyệt đơn < 1 triệu
-   Trưởng phòng: duyệt đơn < 10 triệu
-   Giám đốc: duyệt tất cả

👉 Request sẽ đi qua từng cấp cho đến khi được xử lý.

KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
    ↓
POST /api/orders/approve
    ↓
OrderService
    ↓
EmployeeHandler
    ↓
ManagerHandler
    ↓
DirectorHandler
```

BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n ChainResponsibilityDemo
cd ChainResponsibilityDemo
```

CẤU TRÚC THƯ MỤC

```
ChainResponsibilityDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   ├── OrderService.cs
│   └── Handlers
│       ├── ApprovalHandler.cs
│       ├── EmployeeHandler.cs
│       ├── ManagerHandler.cs
│       └── DirectorHandler.cs
├── Models
│   ├── OrderRequest.cs
│   └── ApprovalResult.cs
└── Program.cs
```

❁ 1. MODEL

Models/OrderRequest.cs

```
namespace ChainResponsibilityDemo.Models;
```

```
public class OrderRequest
{
    public decimal Amount { get; set; }
}
```

Models/ApprovalResult.cs

```
namespace ChainResponsibilityDemo.Models;
```

```
public class ApprovalResult
{
    public bool Approved { get; set; }
    public string ApprovedBy { get; set; }
    public string Message { get; set; }
}
```

❁ 2. ABSTRACT HANDLER (CHAIN CORE)

Services/Handlers/ApprovalHandler.cs

```
using ChainResponsibilityDemo.Models;
```

```
namespace ChainResponsibilityDemo.Services.Handlers;
```

```
public abstract class ApprovalHandler
{
    protected ApprovalHandler _nextHandler;
```

```
        public void SetNext(ApprovalHandler nextHandler)
        {
            _nextHandler = nextHandler;
        }

        public abstract Task<ApprovalResult>
        Handle(OrderRequest request);
    }
}
```

3. CONCRETE HANDLERS

```
EmployeeHandler.cs
using ChainResponsibilityDemo.Models;

namespace ChainResponsibilityDemo.Services.Handlers;

public class EmployeeHandler : ApprovalHandler
{
    public override async Task<ApprovalResult>
    Handle(OrderRequest request)
    {
        if (request.Amount < 1_000_000)
        {
            return new ApprovalResult
            {
                Approved = true,
                ApprovedBy = "Employee",
                Message = "Approved by Employee"
            };
        }

        return await _nextHandler.Handle(request);
    }
}
```

```
ManagerHandler.cs
using ChainResponsibilityDemo.Models;

namespace ChainResponsibilityDemo.Services.Handlers;

public class ManagerHandler : ApprovalHandler
{
    public override async Task<ApprovalResult>
    Handle(OrderRequest request)
```

```

    {
        if (request.Amount < 10_000_000)
        {
            return new ApprovalResult
            {
                Approved = true,
                ApprovedBy = "Manager",
                Message = "Approved by Manager"
            };
        }

        return await _nextHandler.Handle(request);
    }
}

```

DirectorHandler.cs

```

using ChainResponsibilityDemo.Models;

namespace ChainResponsibilityDemo.Services.Handlers;

public class DirectorHandler : ApprovalHandler
{
    public override async Task<ApprovalResult>
    Handle(OrderRequest request)
    {
        return new ApprovalResult
        {
            Approved = true,
            ApprovedBy = "Director",
            Message = "Approved by Director"
        };
    }
}

```

4. ORDER SERVICE (TẠO CHAIN)

Services/OrderService.cs

```

using ChainResponsibilityDemo.Models;
using ChainResponsibilityDemo.Services.Handlers;

namespace ChainResponsibilityDemo.Services;

public class OrderService
{
    private readonly ApprovalHandler _chain;

```

```

public OrderService()
{
    var employee = new EmployeeHandler();
    var manager = new ManagerHandler();
    var director = new DirectorHandler();

    employee.SetNext(manager);
    manager.SetNext(director);

    _chain = employee;
}

public async Task<ApprovalResult>
Approve(OrderRequest request)
{
    return await _chain.Handle(request);
}
}

```

✿ 5. CONTROLLER

```

Controllers/OrdersController.cs
using Microsoft.AspNetCore.Mvc;
using ChainResponsibilityDemo.Models;
using ChainResponsibilityDemo.Services;

namespace ChainResponsibilityDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController(OrderService service)
    {
        _service = service;
    }

    [HttpPost("approve")]
    public async Task<ActionResult> Approve(OrderRequest
request)
    {
        var result = await _service.Approve(request);
    }
}

```

```
        return Ok(result);
    }
}
```

6. ĐĂNG KÝ DI

```
Program.cs
using ChainResponsibilityDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddTransient<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();

app.Run();
```

CHẠY BACKEND

```
dotnet run
```

Test API:

POST /api/orders/approve

```
{
  "amount": 500000
}
```

FRONTEND – REACT

Tạo project

```
npx create-react-app approval-ui
```

```
cd approval-ui
npm install axios
```

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [amount, setAmount] = useState(1000000);
  const [result, setResult] = useState("");

  const approve = async () => {
    const response = await axios.post(
      "https://localhost:xxxx/api/orders/approve",
      { amount }
    );

    setResult(response.data.message);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Order Approval</h2>

      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
      />

      <br /><br />

      <button onClick={approve}>Submit</button>

      <h3>{result}</h3>
    </div>
  );
}

export default App;
```

▶ CHẠY FRONTEND

```
npm start
```

LUỒNG XỬ LÝ

CASE 1: 500000

Employee → xử lý được → STOP

 Output:

Approved by Employee

CASE 2: 5000000

Employee → không xử lý
→ Manager → xử lý → STOP

 Output:

Approved by Manager

CASE 3: 50000000

Employee → skip
Manager → skip
Director → xử lý

 Output:

Approved by Director

KIẾN THỨC SINH VIÊN HỌC ĐƯỢC

- ☒ Hiểu rõ Chain of Responsibility
- ☒ Tách rời logic xử lý theo từng cấp
- ☒ Dễ mở rộng (thêm handler mới không sửa code cũ)
- ☒ So sánh với:
 - if-else dài ✗
 - switch-case ✗
 - polymorphism + chain ☒

BÀI TẬP MỞ RỘNG

1. Thêm cấp:
 - Vice Director (20 triệu)
2. Từ chối nếu:
 - 100 triệu
3. Log lại handler đã xử lý
4. Kết hợp với:
 - Logging middleware
 - Authorization